

To evaluate how MIMIR and LLVM compare on real-world programs with large control flow graphs, we used MIMIR's `regex` plugin to compile a large regular expression matching a single HTTP access-log line, incorporating extensive enumerations of HTTP methods, status codes, timestamps, and path characters. The resulting matcher contains 1058 continuations in MIMIR and the same number of basic blocks when compiled to LLVM. Free variable analysis took $476\mu s$, while LLVM's dominance analysis completed in $\approx 100\mu s$. β -reduction required $9755\mu s$, whereas inlining took $\approx 3800\mu s$ (up to $\approx 9400\mu s$ with optimizations).

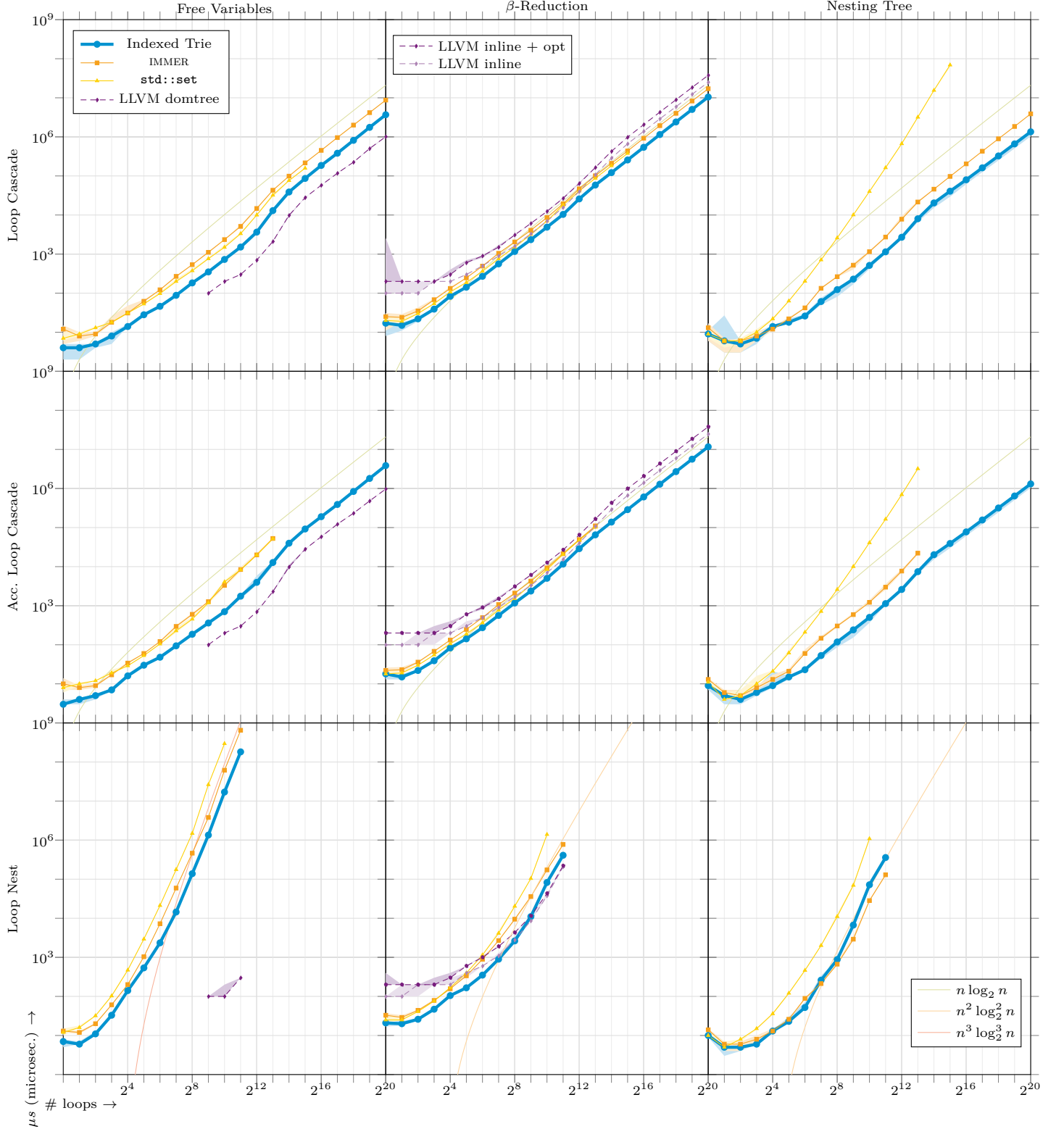


Figure 1: Performance (in μs , lower is better) of three operations—free variable computation, β -reduction, and nesting tree computation—on three synthetic programs with increasing numbers of loops. The plots compare three different set implementations: the indexed trie, IMMER, and `std::set`. Additionally, free variable computation is compared against LLVM’s dominance analysis; β -reduction is compared against LLVM’s inliner, both with and without optimizations that simulate MIMIR’s on-the-fly optimizations. For reference, $10^3 \mu s = 1 \text{ ms}$, $10^6 \mu s = 1 \text{ s}$.